

ЛР #5: [C++ & UNIX]: C++ OOP / PARALLEL

Цель

Познакомить студента с принципами объектно-ориентированного программирования на примере создания сложной синтаксической структуры. Придумать синтаксис своего персонального мини-языка параллельного программирования, а также реализовать его разбор и вычисление.

Задача

1. [C++ PARALLEL LANG] Создать параллельный язык программирования

Требуется создать язык программирования, в котором будет доступна установка следующих команд:

- Установка счетного цикла
- Вывод в консоль
- Вывод в файл в режиме добавления
- Арифметические операции +, -, *, /

Счетный цикл должен поддерживать дальнейшую установку всех остальных поддерживаемых команд.

Для реализации задачи использовать технологию объектно-ориентированного программирования в части реализации поддерживаемых команд языка.

В программе должны быть отражены следующие шаги:

- 1.1. Текстовый ввод команд. Каждая новая строка – это новый набор команд.
- 1.2. Ожидание команды на окончание ввода
- 1.3. Параллельное исполнение введенных строк (наборов команд). Наборы команд должны исполняться параллельно. В консоли фиксировать время запуска / завершения каждого потока. При выводе информации о времени указывать принадлежность потока к строке (набору команд)

```
#include <iostream>
#include <mutex>
#include <vector>
#include <string>
#include <sstream>
#include <fstream>
#include <thread>
#include <chrono>
#include <memory>
#include <iomanip>

std::mutex global_mutex;

class Command {
public:
    virtual ~Command() = default;
    virtual void execute(int set_id) = 0;
};
```

```

class PrintCommand : public Command {
    std::string message;
public:
    PrintCommand(std::string msg) : message(std::move(msg)) {}
    void execute(int set_id) override {
        std::lock_guard<std::mutex> lock(global_mutex);
        std::cout << "[Набор самурая №" << set_id << "][КОНСОЛЬ]: "
<< message << std::endl;
    }
};

class FileAppendCommand : public Command {
    std::string filename;
    std::string data;
public:
    FileAppendCommand(std::string fname, std::string d) :
filename(std::move(fname)), data(std::move(d)) {}
    void execute(int set_id) override {
        std::lock_guard<std::mutex> lock(global_mutex);
        std::ofstream file(filename, std::ios::app);
        if (file.is_open()) {
            file << data << "\n";
            std::cout << "[Набор самурая №" << set_id << "][Файл
самурая]: Дозаписано в '" << filename << "'" << std::endl;
        } else {
            std::cerr << "[Набор самурая №" << set_id << "][Ошибка
ронина]: Не удалось открыть файл '" << filename << std::endl;
        }
    }
};

class MathCommand : public Command {
    double operand1;
    double operand2;
    char operation;
public:
    MathCommand(double op1, char op, double op2) : operand1(op1),
operand2(op2), operation(op) {}
    void execute(int set_id) override {
        double result = 0;
        bool valid = true;
        switch (operation) {
            case '+': result = operand1 + operand2; break;
            case '-': result = operand1 - operand2; break;
            case '*': result = operand1 * operand2; break;

```

```

        case '/':
            if (operand2 != 0) result = operand1 / operand2;
            else { valid = false; }
            break;
        default: valid = false;
    }

    std::lock_guard<std::mutex> lock(global_mutex);
    if (valid) {
        std::cout << "[Набор самурая №" << set_id << "]"[Расчёт
Тактики боя]: "
                << operand1 << " " << operation << " " <<
operand2 << " = " << result << std::endl;
    } else {
        std::cout << "[Набор самурая №" << set_id << "]"[Расчёт
Тактики боя]: Ошибка вычисления исходов боя(" << operand1 <<
operation << operand2 << ")" << std::endl;
    }
}
};

class LoopCommand : public Command {
    int iterations;
    std::vector<std::shared_ptr<Command>> inner_commands;
public:
    LoopCommand(int count) : iterations(count) {}
    void addCommand(std::shared_ptr<Command> cmd) {
        inner_commands.push_back(cmd);
    }

    void execute(int set_id) override {
        for (int i = 0; i < iterations; ++i) {
            {
                std::lock_guard<std::mutex> lock(global_mutex);
                std::cout << "[Набор самурая №" << set_id << "]"[Путь,
длинною в цикл]: Шаг на пути " << i + 1 << " из " << iterations <<
std::endl;
            }
            for (auto& cmd : inner_commands) {
                cmd->execute(set_id);
            }
        }
    }
};

void log_time(const std::string& status, int set_id) {

```

```

auto now = std::chrono::system_clock::now();
    auto time_t_now = std::chrono::system_clock::to_time_t(now);
    auto ms =
std::chrono::duration_cast<std::chrono::milliseconds>(now.time_since_
epoch()) % 1000;
    std::tm tm_buf;
#if defined(_MSC_VER)
    localtime_s(&tm_buf, &time_t_now);
#else
    localtime_r(&time_t_now, &tm_buf);
#endif

    std::lock_guard<std::mutex> lock(global_mutex);
    std::cout << ">>> [" << std::put_time(&tm_buf, "%H:%M:%S") << "."
<< std::setfill('0') << std::setw(3) << ms.count()
        << "]" Поток Набора самурая №" << set_id << " " <<
status << std::endl;
}

void run_set(int set_id, std::vector<std::shared_ptr<Command>>
commands) {
    log_time("ЗАПУЩЕН", set_id);
    for (auto& cmd : commands) {
        cmd->execute(set_id);
    }
    log_time("ЗАБЕПШЕН", set_id);
}

std::shared_ptr<Command> parse_single_command(std::stringstream& ss)
{
    std::string cmd_type;
    if (!(ss >> cmd_type)) return nullptr;

    if (cmd_type == "print_to_screen") {
        std::string msg;
        ss >> msg;
        return std::make_shared<PrintCommand>(msg);
    }
    else if (cmd_type == "file_append") {
        std::string fname, data;
        ss >> fname >> data;
        return std::make_shared<FileAppendCommand>(fname, data);
    }
    else if (cmd_type == "calc") {
        double op1, op2;

```

```

        char sign;
        ss >> op1 >> sign >> op2;
        return std::make_shared<MathCommand>(op1, sign, op2);
    }
    return nullptr;
}

std::vector<std::shared_ptr<Command>> parse_line(const std::string&
line) {
    std::vector<std::shared_ptr<Command>> commands;
    std::stringstream ss(line);
    std::string token;

    while (ss >> token) {
        if (token == "print_to_screen") {
            std::string msg; ss >> msg;
            commands.push_back(std::make_shared<PrintCommand>(msg));
        }
        else if (token == "file_append") {
            std::string fname, data; ss >> fname >> data;

commands.push_back(std::make_shared<FileAppendCommand>(fname, data));
        }
        else if (token == "calc") {
            double op1, op2; char sign;
            ss >> op1 >> sign >> op2;
            commands.push_back(std::make_shared<MathCommand>(op1,
sign, op2));
        }
        else if (token == "loop") {
            int count;
            ss >> count;
            auto loop = std::make_shared<LoopCommand>(count);
            std::string bracket;
            if (ss >> bracket && bracket == "[") {
                std::string inner_token;
                std::string sub_cmd_line = "";
                while (ss >> inner_token && inner_token != "]") {
                    if (inner_token == ";") {
                        std::stringstream sub_ss(sub_cmd_line);
                        auto sub_cmd = parse_single_command(sub_ss);
                        if (sub_cmd) loop->addCommand(sub_cmd);
                        sub_cmd_line = "";
                    } else {
                        sub_cmd_line += inner_token + " ";
                    }
                }
            }
        }
    }
}

```

```

        }
        if (!sub_cmd_line.empty()) {
            std::stringstream sub_ss(sub_cmd_line);
            auto sub_cmd = parse_single_command(sub_ss);
            if (sub_cmd) loop->addCommand(sub_cmd);
        }
    }
    commands.push_back(loop);
}

return commands;
}

int main() {
    std::setlocale(LC_ALL, "Russian");
    std::vector<std::vector<std::shared_ptr<Command>>>
program_storage;
    std::string line;

    std::cout << "_____Интерпретатор 'the last of
progs'_____ " << std::endl;
    std::cout << "Правила ввода команд (в одну строку через
пробел): \n"
        << " - print_to_screen [слово] \n"
        << " - file_append [файл.txt] [слово] \n"
        << " - calc [число] [операция +, -, *, /] [число] \n"
        << " - loop [кол-во] [ команду_1 ; команду_2 ; ]
(обязательно с пробелами вокруг '[', ']' и ';') \n"
        << "Каждая строка — новый независимый набор. \n"
        << "Для завершения ввода нажмите ENTER на пустой
строке. \n" << std::endl;

    int line_counter = 1;
    while (true) {
        std::cout << "Набор " << line_counter << " > ";
        std::getline(std::cin, line);
        if (line.empty()) {
            break;
        }
        program_storage.push_back(parse_line(line));
        line_counter++;
    }

    std::cout << "\n _____ Самурай начал свой путь по выполнению
наборов _____ \n" << std::endl;

```

```

std::vector<std::thread> workers;
for (size_t i = 0; i < program_storage.size(); ++i) {
    workers.emplace_back(run_set, static_cast<int>(i + 1),
program_storage[i]);
}

for (auto& thread : workers) {
    if (thread.joinable()) {
        thread.join();
    }
}

std::cout << "\n_____ " <<
std::endl;
std::cout << "Все потоки завершили работу. Последняя программа
завершила свой путь." << std::endl;
return 0;
}

```

```

vladimir_lamtev@Vladimir:/usr/local/cpp_linux_labs/lab_5/src$ g++
the_last_one.cpp -o main
^[[Avladimir_lamtev@Vladimir:/usr/local/cpp_linux_labs/lab_5/src$ ./main

```

_____Интерпретатор 'the last of progs'_____

Правила ввода команд (в одну строку через пробел):

- print_to_screen [слово]
- file_append [файл.txt] [слово]
- calc [число] [операция +, -, *, /] [число]
- loop [кол-во] [команду_1 ; команду_2 ;] (обязательно с пробелами вокруг '[', ']' и ';')

Каждая строка — новый независимый набор.

Для завершения ввода нажмите ENTER на пустой строке.

Набор 1 > print_to_screen пупу

Набор 2 > calc 228+1337

Набор 3 > file_append aboba.txt amogus

Набор 4 >

_____Самурай начал свой путь по выполнению наборов_____

```

>>> [09:39:19.046] Поток Набора самурая №1 ЗАПУЩЕН
>>> [09:39:19.047] Поток Набора самурая №3 ЗАПУЩЕН
[Набор самурая №3][Файл самурая]: Дозаписано в 'aboba.txt'
>>> [09:39:19.048] Поток Набора самурая №3 ЗАВЕРШЕН
>>> [09:39:19.047] Поток Набора самурая №2 ЗАПУЩЕН
>>> [09:39:19.048] Поток Набора самурая №2 ЗАВЕРШЕН
[Набор самурая №1][КОНСОЛЬ]: пупу

```

>>> [09:39:19.048] Поток Набора самурая №1 ЗАВЕРШЕН

Все потоки завершили работу. Последняя программа завершила свой путь.

vladimir_lamtev@Vladimir:/usr/local/cpp_linux_labs/lab_5/src\$./main

_____Интерпретатор 'the last of progs'_____

Правила ввода команд (в одну строку через пробел):

- print_to_screen [слово]
- file_append [файл.txt] [слово]
- calc [число] [операция +, -, *, /] [число]
- loop [кол-во] [команду_1 ; команду_2 ;] (обязательно с пробелами вокруг '[', ']' и ';')

Каждая строка — новый независимый набор.

Для завершения ввода нажмите ENTER на пустой строке.

Набор 1 > loop 3 [file_append aboba_2.txt тум тумтумтумтумтумтумтуту тутуту там там ;]

Набор 2 >

_____Самурай начал свой путь по выполнению наборов_____

>>> [09:53:00.514] Поток Набора самурая №1 ЗАПУЩЕН

[Набор самурая №1][Путь, длиною в цикл]: Шаг на пути 1 из 3

[Набор самурая №1][Файл самурая]: Дозаписано в 'aboba_2.txt'

[Набор самурая №1][Путь, длиною в цикл]: Шаг на пути 2 из 3

[Набор самурая №1][Файл самурая]: Дозаписано в 'aboba_2.txt'

[Набор самурая №1][Путь, длиною в цикл]: Шаг на пути 3 из 3

[Набор самурая №1][Файл самурая]: Дозаписано в 'aboba_2.txt'

>>> [09:53:00.517] Поток Набора самурая №1 ЗАВЕРШЕН

Все потоки завершили работу. Последняя программа завершила свой путь.

итоговый вид получившихся файлов представлен в файлах txt в папке src

2. [LOG] Результат всех вышеперечисленных шагов сохранить в репозиторий (+ отчет по данной ЛР в папку doc)

Фиксацию ревизий производить строго через ветку dev. С помощью скриптов накатить ревизии на stg и на prd. Скрипты разместить в корне репозитория. Также создать скрипты по возврату к виду текущей ревизии (даже если в папке имеются несохраненные изменения + новые файлы).